

4. Voice Control Muto Color Recognition

Before running this program, you need to bind the port numbers of the voice board and the ROS expansion board on the host machine. You can refer to the previous chapter for binding. When entering the Docker container, you need to mount this voice board so that it can be recognized in the Docker container.

1. Function Description

After the program starts, say "Hello, yahboom" to the module. The module will reply with "I'm here" to wake up the voice board. Then, use the mouse to select a color on the screen (currently recognizable colors are red, green, blue, and yellow), hold it, and then ask, "What color is this?" It will then calculate the HSV value and answer what color it is.

2. Code Reference Path

After entering the Docker container, the source code for this function is located at:

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voice_ctrl/voice_Ctrl_color_identify.py
```

3. Program Startup

3.1. Startup Command

In the first terminal (enter Docker), input:

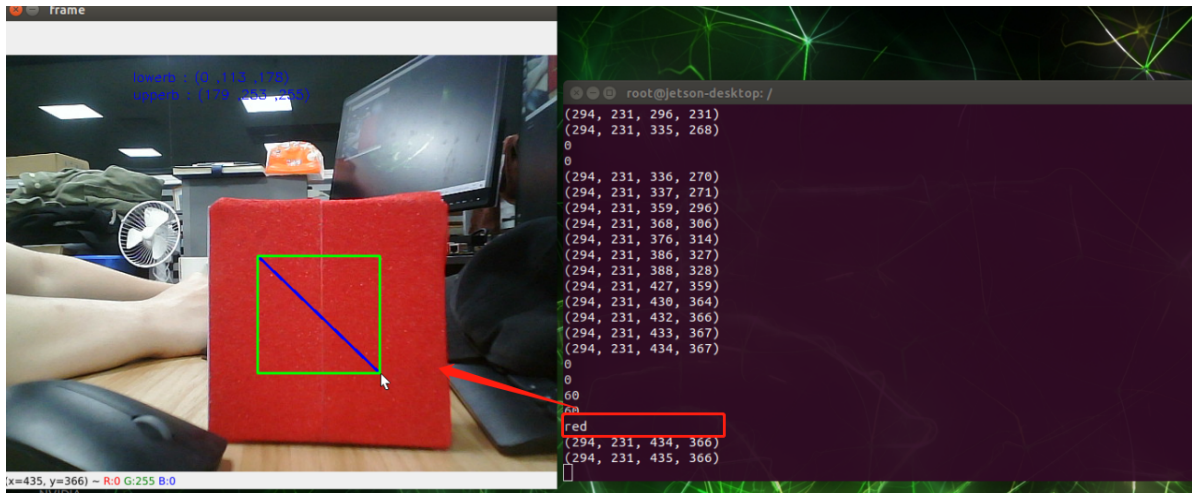
```
cd  
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voice_ctrl/  
python3 voice_Ctrl_color_identify.py
```

In the second terminal (enter Docker), input:

```
ros2 launch yahboom_speech speech.launch.py
```

At this point, we select the specified color in the GUI, **do not release the mouse**, and then ask Muto: "What color is this?"

Muto will tell us the closest color in the current area.



4. Core Code Analysis

The principle of this color recognition is very simple. It judges the color based on the HSV value of the selected area, and then sends the corresponding voice command to the voice board according to the recognition result to announce the recognition result. The code is as follows:

```
def process(self, rgb_img):
    H = [];
    S = [];
    V = [];

    if self.select_flags == True or self.selection_completed == True:
        if self.select_flags == True:
            cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
            cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)
        else:
            cv.rectangle(rgb_img, self.cols, self.rows, (0, 0, 255), 2)

        if self.Roi_init[0] != self.Roi_init[2] and
self.Roi_init[1] != self.Roi_init[3]:
            HSV = cv.cvtColor(rgb_img, cv.COLOR_BGR2HSV)
            for i in range(self.Roi_init[0], self.Roi_init[2]):
                for j in range(self.Roi_init[1], self.Roi_init[3]):
                    H.append(HSV[j, i][0])
                    S.append(HSV[j, i][1])
                    V.append(HSV[j, i][2])

            if len(H) > 0:
                H_min = min(H); H_max = max(H)
                S_min = min(S); S_max = 253
                V_min = min(V); V_max = 255

                lowerb = 'lowerb : (' + str(H_min) + ' , ' + str(S_min) + '
, ' + str(V_min) + ')'
                upperb = 'upperb : (' + str(H_max) + ' , ' + str(S_max) + '
, ' + str(V_max) + ')'
                cv.putText(rgb_img, lowerb, (150, 30),
cv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
                cv.putText(rgb_img, upperb, (150, 50),
cv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)

                command_result = self.voice_command

                if command_result != 999:
```

```

        print(f"Processing command: {command_result}")

    if command_result == 60:
        if H_min == 0 and H_max == 179 :
            self.play_audio(61)
            print("red")
        elif H_min >= 23 and H_min <= 56:
            print("yellow")
            self.play_audio(64)
        elif H_min >= 56 and S_min < 200:
            print("green")
            self.play_audio(63)
        elif H_min >= 60 and S_min >200:
            print("blue")
            self.play_audio(62)

        # Reset command after processing to avoid repeated
actions
        self.voice_command = 999

    return rgb_img

```

Note: Since color recognition is greatly affected by the intensity of light, the HSV parameters calibrated for each color here are not applicable to all scenes. If the recognition effect is not ideal, you need to manually calibrate the parameters yourself. Refer to the course "Chapter 11-2. Control Muto tracking - 4.2. Adjusting HSV Values" and modify the HSV values in the GUI to the corresponding H_min, H_max, etc. of the color.